

 poolside/**Laguna-M.1** 

♥ like 127

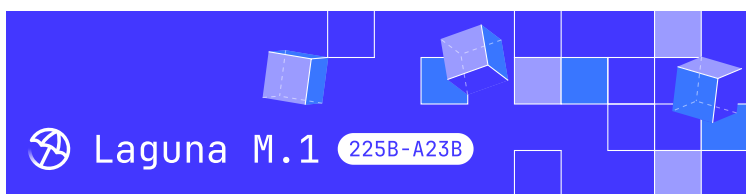
Follow  Poolside 955

 Text Generation  Safetensors vllm laguna laguna-m.1 sglang bf16

 Mixture of Experts conversational custom_code  Eval Results  License: apache-2.0

⋮ → Copy to bucket **NEW**

 **Model card**  Files  xet  Community 8



[Get an API key](#) · [Release blog post](#) · [Technical report](#)

Laguna M.1

Laguna M.1 is a 225B total parameter Mixture-of-Experts model with 23B activated parameters per token designed for agentic coding and long-horizon work.

For more details on how we trained this model, including our Model Factory approach, post-training recipe, async off-policy agent RL, and evaluations, check out our [release blog post](#) and [technical report](#).

Highlights

Downloads last month
2,391



Safetensors

Model size 226B params

Tensor type BF16  Chat template

 Files info

Inference Providers **NEW**

 Text Generation

This model isn't deployed by any Inference Provider.

 4 Ask for provider support

Model tree for poolside/Laguna-M...

Finetunes 3 models

Quantizations    13 models

Spaces using poolside/Laguna... 39

 DeepImagix/self-trained2

 MashaKenzie/vk-ai-bot

- **Large sparse MoE for agentic coding:** Laguna M.1 is a 70-layer MoE transformer with 225B total parameters and 23B activated parameters per token
- **High-capacity expert routing:** After 3 dense SwiGLU layers, Laguna M.1 uses 67 sparse MoE layers with 256 experts, top-k=16 routing and auxiliary-loss-free load balancing
- **Global attention architecture:** Laguna M.1 uses global attention across all layers with 64 Q-heads, 8 KV-heads and softplus attention output gating
- **Native reasoning support:** Interleaved thinking between tool calls with support for enabling and disabling thinking per-request
- **Strong agentic benchmark performance:** Laguna M.1 is competitive with state-of-the-art open-weight and frontier models on SWE-bench Verified, SWE-bench Multilingual, SWE-Bench Pro and Terminal-Bench 2.0
- **Apache 2.0 license:** Use and modify freely for commercial and non-commercial purposes

Model overview

- Training: pre-training, post-training and reinforcement learning stages
- Number of parameters: 225B total with 23B activated per token
- Optimizer: Muon
- Layers: 70 layers with global attention

 Pioneer37/Stickman-Arena

 inchey/coder

 bep40/ml-intern

 abid6160/quizai

 bep40/Space-Mutiverse-Control

 SagarMythos/sunee3 + 31 Spaces

 **Collection including** poolside/Lag...

Laguna M.1  Collection

Our first M-cl... • 4 items • U... • △ 21

Evaluation results

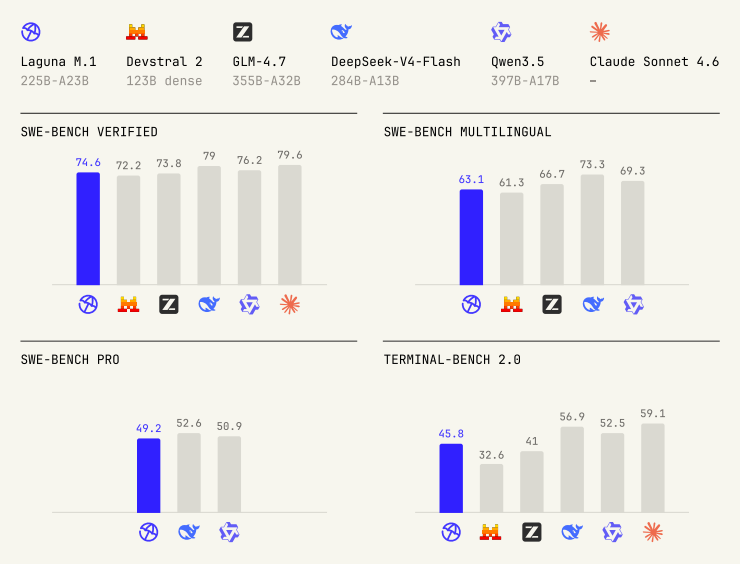
ScaleAI/SWE-bench_Pro · S...   49.2

SWE-bench/SWE-bench_Veri...   74.6

harborframework/terminal-...   45.8

- Experts: 256 experts with 1 shared expert; top-k=16 routing
- Dense layers: first 3 layers are dense SwiGLU; remaining 67 layers are sparse MoE
- Attention: 64 Q-heads, 8 KV-heads, head dimension 128, with softplus attention output gating
- Positional encoding: RoPE with YaRN
- Modality: text-to-text
- Context window: 262,144 tokens
- Reasoning support: interleaved thinking with preserved thinking

Benchmark results



Model	Parameters	SWE-bench Verified	SWE-bench Multilingual	SWE-bench Pro
Laguna M.1	225B-A23B	74.6%	63.1%	49.2%

Model	Parameters	SWE- bench Verified	SWE-bench Multilingual	S b (P Dat
Devstral 2	123B dense	72.2%	61.3%	-
GLM-4.7	355B-A32B	73.8%	66.7%	-
DeepSeek- V4 Flash	284B-A13B	79.0%	73.3%	52.0
Qwen3.5- 397B-A17B	397B-A17B	76.2%	69.3%	50.9
Claude Sonnet 4.6	-	79.6%	-	-

We used the highest publicly-referenced scores for all comparison models across each benchmark. In almost all cases these were official scores published in release blog posts or equivalent, with Claude Sonnet 4.6 shown as a frontier proprietary reference of comparable model size. “-” indicates a score not reported by the model provider.

All benchmarking for Laguna M.1 was completed using our [pool agent harness](#), with a maximum of 500 steps and sandboxed execution. The same sampling parameters were used for all Laguna M.1 benchmarking: temperature=1.0 and top_k=20, with thinking mode enabled and a context length of 256K tokens. All tasks were run in their own sandbox using 8 GB RAM/2 CPUs, with the exception of Terminal-Bench 2.0, which used 48 GB RAM/32 CPUs.

Some base task images and verifiers were patched to fix infrastructure reliability issues inherent in

task setup, such as rate limits on third-party dependencies in external registries used by the verifier. All four agentic benchmarks were run with patched images. We also ran a reward-hack judge post-hoc on Laguna M.1 evaluation runs and did not find significant reward hacking after joint judge review and manual review.

- SWE-bench Verified: mean pass@1 averaged over 4 runs
- SWE-bench Multilingual: mean pass@1 averaged over 4 runs
- SWE-Bench Pro: mean pass@1 averaged over 4 runs
- Terminal-Bench 2.0: mean pass@1 averaged over 4 runs; 48 GB RAM/32 CPUs

Usage

Laguna M.1 has upstream support in vLLM, SGLang, and Transformers, and TRT-LLM thanks to the support of the team at NVIDIA.

pool

pool is a lightweight terminal-based coding agent and a dual Agent Client Protocol client-server.

Download and install for macOS and Linux:

```
curl -fsSL https://downloads.poolside.ai/pool
```

Launch and *Log in with Poolside* to get a free API key.

```
pool
```

Use in any [ACP client](#). Configure Zed and JetBrains automatically:

```
pool acp setup --editor zed|jetbrains
```

Feedback and issues

Submit feedback with `/feedback` and read the [full documentation on GitHub](#).

Deployment

vLLM

Serve Laguna M.1 locally with vLLM and query it from any OpenAI-compatible client (see [Controlling reasoning](#) for tool calls, streaming, and reasoning extraction):

Laguna support landed in vLLM via [vllm-project/vllm#41129](#) (shared with [Laguna XS.2](#)) and is available in vLLM 0.21.0 and later.

```
pip install 'vllm>=0.21.0'
```

```
vllm serve \
  --model poolside/Laguna-M.1 \
  --tool-call-parser poolside_v1 \
  --reasoning-parser poolside_v1 \
  --enable-auto-tool-choice \
  --served-model-name laguna \
  --default-chat-template-kwarg '{"enable
```

See the [vLLM recipes page](#) for our Laguna XS.2 model with which the implementation is shared for additional deployment guidance. FP8 and NVFP4 quantized checkpoints are available at [Laguna-M.1-](#)

FP8 and Laguna-M.1-NVFP4; quantization is detected automatically from `quantization_config`, so the same command works with the model ID substituted.

SGLang

Laguna M.1 can be served with SGLang using its OpenAI-compatible server, including support for tool calling, streaming responses, and reasoning parsing:

Laguna support was added to SGLang in [sgl-project/sglang#24204](#). The integration is shared with [Laguna XS.2](#) and is currently available on SGLang main.

```
# Laguna M.1 support is currently on SGLang main
git clone https://github.com/sgl-project/sglang
cd sglang
pip install -e "python[all]"
```

```
sglang serve \
  --trust-remote-code \
  --model-path poolside/Laguna-M.1 \
  --tool-call-parser poolside_v1 \
  --reasoning-parser poolside_v1 \
  --tp 8 \
  --host 0.0.0.0
```

Quantized Laguna M.1 checkpoints are also available as Laguna-M.1-FP8 and Laguna-M.1-NVFP4. SGLang reads the checkpoint `quantization_config`, so you can use the same launch command after replacing the model ID. For more SGLang-specific deployment details, see the [SGLang Cookbook](#).

Transformers

Laguna is supported in Transformers v5.7.0 and later ([huggingface/transformers#45673](https://huggingface.co/transformers#45673)).

Laguna M.1 is a 225B-parameter model; loading the BF16 checkpoint in Transformers requires substantial multi-GPU memory (device_map="auto" shards across available devices). For single-node serving, vLLM or SGLang is recommended.

```
import torch
from transformers import AutoModelForCausalLM

model_id = "poolside/Laguna-M.1"

tokenizer = AutoTokenizer.from_pretrained(model_id)
model = AutoModelForCausalLM.from_pretrained(
    model_id,
    dtype=torch.bfloat16,
    device_map="auto",
)

messages = [
    {"role": "user", "content": "Write a Python function to calculate the sum of the first 100 natural numbers."}
]

# Reasoning is on by default; pass enable_thinking=False to disable it.
inputs = tokenizer.apply_chat_template(
    messages,
    add_generation_prompt=True,
    return_tensors="pt",
    enable_thinking=True,
).to(model.device)

outputs = model.generate(inputs, max_new_tokens=100)
print(tokenizer.decode(outputs[0][inputs.shape[0]:]))
```

TRT-LLM

Laguna is supported in TensorRT-LLM thanks to the team at NVIDIA — model support landed in [NVIDIA/TensorRT-LLM#13559](#), with partial-RoPE fusion added in [#15110](#). Build TensorRT-LLM from a main that includes these PRs (or a release once they ship).

```
from tensorrt_llm import LLM, SamplingParams

llm = LLM(model="poolside/Laguna-M.1", trust_
sampling = SamplingParams(max_tokens=1024, to
out = llm.generate(["Write a Python retry wr
print(out[0].outputs[0].text)
```

If your TensorRT-LLM build pins transformers < 4.58, configuration_laguna.py needs a small compat shim; use the laguna_minimal_overlay.sh helper from the support PR and load TRT-LLM against the overlay directory.

Quantization is detected automatically from quantization_config, so the same recipe works for the [FP8](#) and [NVFP4](#) variants with no extra flags.

Controlling reasoning

Laguna M.1 has native reasoning support and is designed to work best with *preserved thinking*, where reasoning content from prior assistant messages is preserved in the message history. This model will generally reason before calling tools and between tool calls.

```

import json
from openai import OpenAI

client = OpenAI(
    base_url="https://inference.poolside.ai/v1",
    api_key="...",
)

model = "poolside/laguna-m.1"

tools = [{"type": "function", "function": {
    "name": "shell",
    "description": "Execute a bash command and",
    "parameters": {"type": "object", "properties": {}
}}]}

messages = [
    {"role": "system", "content": "You are a code interpreter."},
    {"role": "user", "content": "Run 'uname -a'."}
]

# Thinking is enabled by default when the server is configured for it.
# When using the Poolside API (https://inference.poolside.ai/v1),
response = client.chat.completions.create(
    model=model,
    messages=messages,
    tools=tools,
    stream=True,
)

reasoning, content, tool_calls = "", "", []
for chunk in response:
    delta = chunk.choices[0].delta
    if hasattr(delta, "reasoning_content") and delta.reasoning_content:
        reasoning += delta.reasoning_content
    if hasattr(delta, "content") and delta.content:
        content += delta.content
    if hasattr(delta, "tool_calls") and delta.tool_calls:
        for tc in delta.tool_calls:
            if tc.index >= len(tool_calls):
                tool_calls.append({"id": tc.id, "function": tc.function})

```

```

        if tc.function.name:
            tool_calls[tc.index]["function"]["name"] = tc.function.name
        if tc.function.arguments:
            tool_calls[tc.index]["function"]["arguments"] = tc.function.arguments

print(f"Reasoning: {reasoning}\nContent: {content}")

# Return reasoning in the next request for better reasoning
messages.append({
    "role": "assistant",
    "content": content,
    "reasoning_content": reasoning,
    "tool_calls": [{ "id": tc["id"], "type": "function", "function": {
        "name": tc.function.name,
        "arguments": tc.function.arguments
    }}]
})

messages.append({
    "role": "tool",
    "tool_call_id": tool_calls[0]["id"],
    "content": json.dumps({"stdout": "Darwin a:
})

response = client.chat.completions.create(
    model=model,
    messages=messages,
    tools=tools,
    stream=True,
)

reasoning, content = "", ""
for chunk in response:
    delta = chunk.choices[0].delta
    if hasattr(delta, "reasoning_content") and delta.reasoning_content:
        reasoning += delta.reasoning_content
    if hasattr(delta, "content") and delta.content:
        content += delta.content

print(f"Reasoning: {reasoning}\nContent: {content}")

```

Disabling reasoning

You can disable thinking by setting `enable_thinking` to `False` in a request or by not providing `--default-chat-template-kwarg` `{"enable_thinking": True}` or equivalent when starting the server.

```
from openai import OpenAI
client = OpenAI()

completion = client.chat.completions.create(
    model="poolside/laguna-m.1",
    messages=[
        {"role": "user", "content": "Write a ret:
    ],
    extra_body={
        "chat_template_kwargs": { "enable_thinki
    },
    stream=True
)

for chunk in completion:
    print(chunk.choices[0].delta)
```

For agentic coding use cases, we recommend enabling thinking and preserving reasoning in message history as outlined in the [Controlling reasoning](#) section.

License

This model is licensed under the [Apache 2.0 License](#).

Intended and Responsible Use

Laguna M.1 is designed for software engineering and agentic coding use cases, and you are responsible for confirming that it is appropriate for your



Apache 2.0 License, and should be used consistently with Poolside's Acceptable Use Policy. We advise against circumventing Laguna M.1 safety guardrails without implementing substantially equivalent mitigations appropriate for your use case.

Please report security vulnerabilities or safety concerns to security@poolside.ai.